Generative Adversarial Nets

A Detailed Mathematical and Conceptual Report on Goodfellow et al. (2014)

Prepared from the original GAN research paper

August 2026

Contents

Source and Scope

This report explains the original paper *Generative Adversarial Nets* by Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. The paper introduced the adversarial-net framework in 2014 as a way of learning generative models through a two-player game between a generator and a discriminator. The original paper states that the generator learns to capture the data distribution, while the discriminator estimates whether a sample came from the training distribution or the generator. The framework is formulated as a minimax game and, in the idealized infinite-capacity setting, has its global solution at $p_g = p_{\text{data}}$ and $D(x) = 1/2$ everywhere.

The report follows the paper's main organization: motivation and related work, the adversarial framework, the minimax objective, the minibatch training algorithm, theoretical results, the experiments on MNIST, the Toronto Face Database (TFD), and CIFAR-10, and the stated advantages and disadvantages.

Chapter 1

Introduction

1.1 The problem addressed by the paper

A generative model attempts to learn how observations are distributed. Let

$$x \sim p_{\text{data}}(x)$$

denote an observation from the unknown data-generating distribution.

The goal is to construct a model distribution

$$p_g(x)$$

that resembles $p_{\text{data}}(x)$ and from which new samples can be generated.

The original paper motivates GANs partly by the difficulty of training deep generative models using maximum likelihood. Maximum-likelihood approaches can require difficult probabilistic computations, while the adversarial framework provides an alternative estimation procedure.

1.2 The central idea

The paper introduces two models:

$$\boxed{\text{Generator } G} \qquad \text{and} \qquad \boxed{\text{Discriminator } D}.$$

The generator receives a random latent vector

$$z \sim p_z(z)$$

and transforms it into a synthetic sample:

$$\boxed{x = G(z)}.$$

The discriminator receives either a real sample or a generated sample and outputs a scalar interpreted as the probability that the input is real:

$$\boxed{D(x) \in [0, 1].}$$

Thus:

$$\boxed{z \sim p_z \longrightarrow G(z) \longrightarrow D(G(z)).}$$

At the same time:

$$\boxed{x \sim p_{\text{data}} \longrightarrow D(x).}$$

The discriminator tries to distinguish these two sources, while the generator tries to make generated samples indistinguishable from real samples.

1.3 Counterfeiter–police analogy

The paper uses an intuitive analogy.

The generator can be viewed as a counterfeiter trying to produce convincing fake currency. The discriminator can be viewed as the police trying to detect counterfeit currency.

The generator improves because the discriminator exposes weaknesses in generated samples. The discriminator improves because the generator continually produces new counterexamples.

The desired endpoint is:

$$\boxed{\text{generated samples are indistinguishable from real samples.}}$$

Chapter 2

Why GANs Are Different from Earlier Generative Models

2.1 Likelihood-based generative modeling

Traditional generative modeling often attempts to define an explicit probability model

$$p_\theta(x)$$

and optimize a likelihood-related objective.

However, in complex high-dimensional models, exact likelihood computation or inference can be difficult.

The GAN framework avoids directly requiring an explicit tractable density for the generated distribution.

2.2 Implicit distribution

The generator defines a distribution indirectly.

Sample

$$z \sim p_z(z)$$

and compute

$$x = G(z).$$

The resulting random variable $G(z)$ induces a distribution p_g over the sample space.

Therefore the generator does not necessarily provide a convenient explicit formula for

$$p_g(x).$$

Instead, it provides a procedure for sampling from p_g.

2.3 Why this matters

The generator can therefore be viewed as an implicit generative model:

$$\boxed{\text{latent noise} \rightarrow \text{generator} \rightarrow \text{sample.}}$$

The discriminator supplies the training signal needed to improve this implicit distribution.

Chapter 3

The GAN Architecture

3.1 Generator

Let

$$z \sim p_z(z)$$

be sampled from a simple prior, such as a standard normal or another convenient noise distribution.

The generator is a parameterized function:

$$\boxed{G(z; \theta_g).}$$

The induced generated distribution is denoted by

$$p_g.$$

Different values of z produce different samples:

$$z_1 \to G(z_1), \qquad z_2 \to G(z_2), \qquad z_3 \to G(z_3).$$

Thus the latent variable provides the source of variation.

3.2 Discriminator

The discriminator is another parameterized function:

$$\boxed{D(x; \theta_d).}$$

It outputs a value between zero and one:

$$D(x) \in [0, 1].$$

Interpretation:

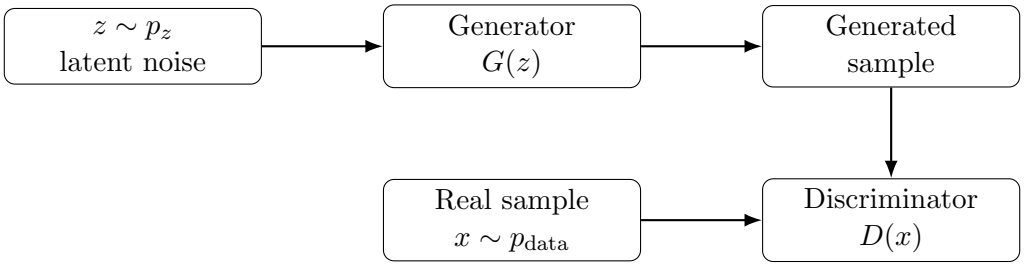
$$D(x) \approx 1 \quad \Rightarrow \quad \text{likely real,}$$

while

$$D(x) \approx 0 \quad \Rightarrow \quad \text{likely generated.}$$

3.3 The complete game

The complete architecture can be summarized as



The discriminator receives both types of examples:

$$x \sim p_{\text{data}} \qquad \text{and} \qquad G(z),\ z \sim p_z.$$

Chapter 4

The Minimax Objective

4.1 The value function

The original GAN objective is

$$\boxed{\min_G \max_D V(D,G)}$$

where

$$\boxed{V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))].}$$

This single equation describes the adversarial game.

4.2 What the discriminator maximizes

For real samples,

$$\log D(x)$$

is large when

$$D(x) \to 1.$$

For generated samples,

$$\log(1 - D(G(z)))$$

is large when

$$D(G(z)) \to 0.$$

Therefore the discriminator maximizes

$$\boxed{\log D(x) + \log(1 - D(G(z))).}$$

In words:

$$\boxed{D: \quad \text{real} \to 1, \qquad \text{fake} \to 0.}$$

4.3 What the generator minimizes

Under the minimax formulation, the generator minimizes

$$\log(1 - D(G(z))).$$

Its goal is therefore to make

$$D(G(z))$$

large, because the discriminator should classify generated samples as real.

Conceptually:

$$\boxed{G: \quad \text{make fake samples look real.}}$$

4.4 Why this is a game

The two objectives are opposing:

$$\boxed{D \text{ tries to distinguish}}$$

while

$$\boxed{G \text{ tries to fool.}}$$

This is fundamentally different from a conventional single-network loss where one model directly minimizes a fixed target.

Chapter 5

Interpreting the Discriminator as a Classifier

The discriminator's objective can be interpreted as binary classification.

Define a label

$$y = \begin{cases} 1, & x \sim p_{\text{data}}, \\ 0, & x \sim p_g. \end{cases}$$

Then

$$D(x) \approx P(y = 1 \mid x).$$

The discriminator maximizes the binary log-likelihood:

$$\boxed{\mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{x \sim p_g} \log(1 - D(x)).}$$

Therefore a GAN contains a conventional classification problem inside the larger generative game.

The important difference is that the negative examples are not fixed. They are continually produced by the generator.

Chapter 6

Training Procedure

6.1 Alternating optimization

The original paper does not optimize G and D simultaneously in one ordinary gradient descent step.

Instead, training alternates:

$$\boxed{\text{update } D \quad \longrightarrow \quad \text{update } G \quad \longrightarrow \quad \text{update } D \quad \longrightarrow \cdots}$$

The paper's Algorithm 1 uses k discriminator updates followed by one generator update. In the experiments, the authors use

$$\boxed{k = 1}.$$

6.2 Discriminator update

Sample a minibatch of real data:

$$\{x^{(1)}, \ldots, x^{(m)}\} \sim p_{\text{data}}.$$

Sample latent vectors:

$$\{z^{(1)}, \ldots, z^{(m)}\} \sim p_z.$$

The generated samples are

$$G(z^{(1)}), \ldots, G(z^{(m)}).$$

The discriminator is updated by ascending the stochastic gradient of

$$\boxed{\frac{1}{m} \sum_{i=1}^{m} \left[\log D(x^{(i)}) + \log \left(1 - D(G(z^{(i)})) \right) \right].}$$

6.3 Generator update

The generator is then updated using

$$\boxed{\frac{1}{m} \sum_{i=1}^{m} \log \left(1 - D(G(z^{(i)})) \right)}$$

under the original minimax formulation.

The paper also introduces a practically useful alternative, discussed later.

6.4 Algorithmic summary

1. Sample real examples from p_{data}.
2. Sample noise vectors from p_z.
3. Generate fake examples using G.
4. Update D to classify real and generated examples correctly.
5. Sample fresh noise.
6. Update G to make the discriminator classify generated examples as real.
7. Repeat.

Chapter 7

Why the Generator's Original Loss Can Saturate

The paper identifies an important optimization problem.

The original minimax generator objective is

$$\min_G \mathbb{E}_z \left[\log(1 - D(G(z)))\right].$$

Early in training, the generator may be very poor. The discriminator can then become highly confident:

$$D(G(z)) \approx 0.$$

Although the generator's objective is large in this situation, its gradient can become weak because the logarithm and sigmoid discriminator can enter a saturated regime.

Thus the generator may receive an insufficient learning signal.

7.1 The non-saturating generator objective

The paper proposes instead maximizing

$$\boxed{\log D(G(z))}$$

or equivalently minimizing

$$\boxed{-\log D(G(z)).}$$

Therefore the practical generator loss can be written as

$$\boxed{L_G^{\mathrm{NS}} = -\mathbb{E}_{z\sim p_z}[\log D(G(z))].}$$

The important statement from the paper is that this alternative has the same fixed point but provides stronger gradients early in training.

Chapter 8

Optimal Discriminator

This is the first major theoretical result.

8.1 Fix the generator

Suppose G is fixed.

Then p_g is fixed, and the discriminator maximizes

$$V(D, G).$$

Because generated samples have distribution p_g, rewrite the objective as

$$V(D, G) = \int p_{\text{data}}(x) \log D(x)\, dx + \int p_g(x) \log(1 - D(x))\, dx.$$

Combine the terms:

$$\boxed{V(D, G) = \int \left[p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)) \right] dx.}$$

8.2 Pointwise maximization

For a fixed x, define

$$a = p_{\text{data}}(x), \qquad b = p_g(x).$$

We need to maximize

$$f(D) = a \log D + b \log(1 - D).$$

Differentiate:

$$\frac{df}{dD} = \frac{a}{D} - \frac{b}{1 - D}.$$

Set the derivative to zero:

$$\frac{a}{D} = \frac{b}{1 - D}.$$

Cross-multiply:

$$a(1 - D) = bD.$$

Therefore:

$$a = aD + bD$$

and

$$D(a + b) = a.$$

Hence

$$\boxed{D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.}$$

This is Proposition 1 of the paper.

8.3 Interpretation

The optimal discriminator compares the density of real data with the density of generated data.

If

$$p_{\text{data}}(x) \gg p_g(x),$$

then

$$D_G^*(x) \approx 1.$$

If

$$p_g(x) \gg p_{\text{data}}(x),$$

then

$$D_G^*(x) \approx 0.$$

If

$$p_{\text{data}}(x) = p_g(x),$$

then

$$\boxed{D_G^*(x) = \frac{1}{2}.}$$

Chapter 9

Virtual Training Criterion

Substitute the optimal discriminator into the value function.

Define

$$\boxed{C(G) = \max_D V(D, G) = V(D^*_G, G).}$$

Then

$$C(G) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \right] + \mathbb{E}_{x \sim p_g} \left[\log \frac{p_g(x)}{p_{\text{data}}(x) + p_g(x)} \right].$$

This expression allows the generator problem to be analyzed directly in distribution space.

Chapter 10

Connection to Jensen–Shannon Divergence

Define the mixture distribution

$$m(x) = \frac{1}{2}\left(p_{\text{data}}(x) + p_g(x)\right).$$

Then

$$p_{\text{data}}(x) + p_g(x) = 2m(x).$$

Substituting:

$$C(G) = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}(x)}{2m(x)}\right] + \mathbb{E}_{p_g}\left[\log \frac{p_g(x)}{2m(x)}\right].$$

Split the logarithms:

$$C(G) = -\log 4 + D_{\text{KL}}(p_{\text{data}} \| m) + D_{\text{KL}}(p_g \| m).$$

By definition,

$$\boxed{\text{JSD}(p_{\text{data}} \| p_g) = \frac{1}{2} D_{\text{KL}}(p_{\text{data}} \| m) + \frac{1}{2} D_{\text{KL}}(p_g \| m).}$$

Therefore:

$$\boxed{C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \| p_g).}$$

This is the central theoretical connection of the original GAN paper.

10.1 Why this is important

The Jensen–Shannon divergence satisfies

$$\text{JSD}(p_{\text{data}} \| p_g) \geq 0$$

and

$$\text{JSD}(p_{\text{data}} \| p_g) = 0 \iff p_{\text{data}} = p_g.$$

Therefore

$$C(G) \geq -\log 4.$$

The minimum is achieved exactly when

$$\boxed{p_g = p_{\text{data}}.}$$

At this point:

$$\boxed{D_G^*(x) = \frac{1}{2}.}$$

Thus the ideal GAN equilibrium is:

$$\boxed{p_g = p_{\text{data}}, \qquad D(x) = \frac{1}{2}.}$$

Chapter 11

Theorem 1: Global Optimality

The paper's Theorem 1 states that the global minimum of the virtual training criterion

$$C(G)$$

is achieved if and only if

$$p_g = p_{\text{data}}.$$

At that point:

$$\boxed{C(G) = -\log 4.}$$

11.1 Step-by-step proof

We have already obtained

$$C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \| p_g).$$

Since

$$\text{JSD}(p_{\text{data}} \| p_g) \geq 0,$$

we have

$$C(G) \geq -\log 4.$$

Equality occurs exactly when

$$\text{JSD}(p_{\text{data}} \| p_g) = 0.$$

Therefore:

$$\text{JSD}(p_{\text{data}} \| p_g) = 0 \iff p_{\text{data}} = p_g.$$

Hence:

$$\boxed{C(G)_{\min} = -\log 4}$$

and

$$\boxed{p_g = p_{\text{data}}}$$

is the unique distribution-level optimum.

11.2 Meaning of the theorem

The theorem does not say that every finite neural-network training run will automatically reach this solution.

It says that in the idealized non-parametric setting, if the model family is sufficiently expressive and optimization reaches the theoretical optimum, the target distribution is exactly the data distribution.

This distinction between the ideal distribution-level result and practical neural-network optimization is important.

Chapter 12

Proposition 2: Convergence

The paper also gives a convergence proposition.

Suppose:

1. G and D have sufficient capacity;
2. at every stage the discriminator can reach its optimum for the current generator;
3. the generator is updated to improve the resulting criterion.

Then the generated distribution converges toward

$$\boxed{p_g = p_{\text{data}}.}$$

This result provides the theoretical motivation for alternating optimization.

12.1 Practical qualification

The theorem is an idealized result.

In an actual neural network:

$$G = G_{\theta_g}, \qquad D = D_{\theta_d},$$

and both parameter spaces are finite-dimensional and non-convex.

Therefore:

$$\boxed{\text{theoretical global optimum} \neq \text{guaranteed practical convergence.}}$$

The actual training dynamics depend on network capacity, optimization, learning rates, mini-batches, initialization, and how well D and G are synchronized.

Chapter 13

Why the GAN Objective Is Not Ordinary Maximum Likelihood

GANs do not directly optimize

$$\log p_g(x)$$

for observed training points.

Instead, they optimize an adversarial criterion involving a discriminator.

This is why GANs are commonly described as implicit generative models.

The generator is trained through:

$$z \to G(z) \to D(G(z)) \to \text{gradient for } G.$$

The discriminator supplies the learning signal.

13.1 No explicit density required

The generator only needs to transform noise into samples:

$$z \sim p_z \quad \Rightarrow \quad G(z).$$

It is not necessary to evaluate

$$p_g(x)$$

explicitly for every generated sample.

This was one of the important motivations for the framework.

Chapter 14

Experimental Setup

The paper evaluates adversarial nets on:

MNIST

Toronto Face Database (TFD)

CIFAR-10.

14.1 Generator architecture

The paper reports generator networks using a mixture of:

- rectifier linear activations;
- sigmoid activations.

Noise is supplied as the input to the bottommost layer of the generator in the reported experiments.

14.2 Discriminator architecture

The discriminator uses maxout activations.

Dropout is applied during discriminator training.

The paper also studies convolutional/deconvolutional architectures for CIFAR-10 and reports that the convolutional discriminator and deconvolutional generator produce improved visual results relative to the fully connected setup.

Chapter 15

Evaluating GANs

15.1 The difficulty of likelihood evaluation

A major difficulty is that GANs do not directly provide an explicit likelihood

$$p_g(x).$$

The authors therefore use a Gaussian Parzen-window estimator to approximate the density of generated samples.

15.2 Parzen-window idea

Suppose generated samples are

$$x_1, \ldots, x_N.$$

A Gaussian kernel estimator can be constructed schematically as

$$\hat{p}(x) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{N}(x; x_i, \sigma^2 I).$$

The resulting approximate density is evaluated on held-out test examples.

The paper explicitly notes that this method has relatively high variance and does not perform well in high-dimensional spaces, but it was used because exact likelihood is unavailable.

Chapter 16

Experimental Results

16.1 MNIST and TFD

The paper reports the following Parzen-window log-likelihood estimates:

Table 16.1: Parzen-window log-likelihood estimates reported in the original GAN paper. Higher values are better within this evaluation.

Model	MNIST	TFD
DBN	138 ± 2	1909 ± 66
Stacked CAE	121 ± 1.6	2110 ± 50
Deep GSN	214 ± 1.1	1890 ± 29
Adversarial nets	$\mathbf{225 \pm 2}$	2057 ± 26

On MNIST, the adversarial model obtains the highest reported value among the listed models. On TFD, it is competitive but does not exceed the stacked CAE result.

16.2 Important caution about the metric

The paper itself points out limitations of Parzen-window estimation, especially its variance and poor behavior in high-dimensional spaces.

Therefore the reported likelihood values should not be interpreted as an exact likelihood evaluation of the GAN.

16.3 Qualitative evaluation

The experiments also display generated samples from:

- MNIST;
- TFD;
- CIFAR-10.

The paper compares generated samples with nearby training examples to investigate whether the generator merely memorizes the training set.

The generated samples demonstrate that the adversarial framework can learn recognizable structures from random latent vectors.

Chapter 17

Latent Space and Interpolation

A useful property of the generator is that different points in latent space can generate different samples.

Suppose

$$z_1, z_2 \sim p_z.$$

Consider the linear interpolation

$$\boxed{z(\lambda) = (1-\lambda)z_1 + \lambda z_2, \qquad 0 \leq \lambda \leq 1.}$$

Then generate:

$$G(z(\lambda)).$$

The paper reports smooth transitions in latent space, demonstrating that the generator can learn structured transformations between samples.

Conceptually:

$$z_1 \to G(z_1)$$

and

$$z_2 \to G(z_2)$$

can be connected by a continuous path through latent space.

Chapter 18

Advantages of the GAN Framework

The original paper highlights several advantages.

18.1 No Markov chains

GAN training and generation do not require the Markov-chain sampling procedures associated with certain earlier generative models.

Generation is simply:

$$z \sim p_z \quad \rightarrow \quad G(z).$$

18.2 No unrolled approximate inference

The framework does not require an unrolled inference network during generation.

18.3 Backpropagation-based training

When G and D are multilayer perceptrons, the complete system can be trained using backpropagation.

18.4 Sharp generated samples

The adversarial objective directly encourages generated samples to become difficult for the discriminator to distinguish from real data. This can lead to sharp and realistic samples.

18.5 Flexible generator architecture

The theoretical framework does not require a particular generator architecture. In practice, different neural architectures can be chosen according to the data domain.

Chapter 19

Disadvantages and Difficulties

19.1 No explicit likelihood

The original GAN formulation does not directly provide a convenient explicit value of

$$p_g(x).$$

This makes likelihood-based evaluation difficult.

19.2 Training instability

The generator and discriminator must be trained in a coordinated manner.

If one becomes much stronger than the other, the useful learning signal can deteriorate.

19.3 Mode collapse

A generator may produce samples from only a subset of the modes of the data distribution.

Conceptually, if

$$pdata$$

contains many different valid regions but

$$pg$$

covers only a small subset, the generator can still fool the discriminator on those regions without representing the full distribution.

This is commonly called

$$\boxed{\text{mode collapse}}.$$

The original paper discusses the challenge of maintaining sufficient diversity and synchronizing the generator and discriminator.

19.4 Non-convex optimization

With neural networks,

$$\theta_g, \theta_d$$

are learned through a non-convex optimization problem.

The theoretical global optimum therefore does not imply that standard gradient-based training will always find it.

Chapter 20

GAN versus VAE

Both GANs and VAEs are generative models, but their mathematical structures differ.

Table 20.1: Conceptual comparison of VAE and GAN

Property	**VAE**	**GAN**
Core mechanism	Latent-variable probabilistic model	Generator–discriminator game
Likelihood	Explicit likelihood framework	Not directly likelihood-based
Latent representation	Explicit encoder-based representation	Usually implicit generator input
Training objective	ELBO / likelihood-related	Adversarial minimax objective
Sample quality	Can be smoother	Often sharp
Coverage	Reconstruction and KL encourage broad representation	Can suffer from mode collapse
Inference	Encoder available	Original GAN has no encoder
Generation	Sample latent z, decode	Sample z, apply G

The supplied study material also describes the VAE–GAN distinction in these terms: VAEs are explicit latent-variable probabilistic models, while GANs use a generator–discriminator game and can produce sharper samples but may fail to cover all modes.

Chapter 21

The Complete Mathematical Picture

The GAN can be understood through the following chain.

21.1 Step 1: choose a simple latent prior

$$\boxed{z \sim p_z(z)}$$

21.2 Step 2: generate a sample

$$\boxed{x_{\text{fake}} = G(z)}$$

The generator induces

$$p_g.$$

21.3 Step 3: discriminate

For a real sample:

$$D(x_{\text{real}}) \to 1.$$

For a generated sample:

$$D(x_{\text{fake}}) \to 0.$$

21.4 Step 4: update the discriminator

Maximize:

$$\boxed{\mathbb{E}_{p_{\text{data}}} \log D(x) + \mathbb{E}_{p_z} \log(1 - D(G(z))).}$$

21.5 Step 5: update the generator

Under the original minimax objective:

$$\boxed{\min_G \mathbb{E}_{p_z}[\log(1 - D(G(z)))].}$$

For the practical non-saturating objective:

$$\boxed{\min_G -\mathbb{E}_{p_z}[\log D(G(z))].}$$

21.6 Step 6: reach the ideal equilibrium

At the distribution-level optimum:

$$\boxed{p_g = p_{\text{data}}.}$$

The optimal discriminator then becomes:

$$\boxed{D(x) = \frac{1}{2}.}$$

The value function reaches:

$$\boxed{V(D^*, G^*) = -\log 4.}$$

Chapter 22

The Most Important Derivation in One Chain

The entire theory can be condensed into one mathematical sequence.

Start with:

$$V(D,G) = \mathbb{E}_{p_{\text{data}}} \log D(x) + \mathbb{E}_{p_g} \log(1 - D(x)).$$

For fixed G:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substitute:

$$C(G) = V(D_G^*, G).$$

Define

$$m(x) = \frac{p_{\text{data}}(x) + p_g(x)}{2}.$$

Then:

$$\boxed{C(G) = -\log 4 + D_{\text{KL}}(p_{\text{data}} \| m) + D_{\text{KL}}(p_g \| m).}$$

Since:

$$\text{JSD}(p_{\text{data}} \| p_g) = \frac{1}{2} D_{\text{KL}}(p_{\text{data}} \| m) + \frac{1}{2} D_{\text{KL}}(p_g \| m),$$

we obtain:

$$\boxed{C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \| p_g).}$$

Since:

$$\text{JSD}(p_{\text{data}} \| p_g) \geq 0,$$

the minimum occurs at:

$$\boxed{p_{\text{data}} = p_g.}$$

Then:

$$\boxed{D_G^*(x) = \frac{1}{2}.}$$

This is the central mathematical argument of the original paper.

Chapter 23

A Simple Numerical Interpretation

Suppose at a particular point x:

$$pdata(x) = 0.8, \qquad pg(x) = 0.2.$$

Then the optimal discriminator is

$$D^*(x) = \frac{0.8}{0.8 + 0.2} = 0.8.$$

So the discriminator considers the point highly likely to be real.

Now suppose:

$$pdata(x) = pg(x) = 0.5.$$

Then:

$$D^*(x) = \frac{0.5}{0.5 + 0.5} = 0.5.$$

The discriminator cannot distinguish the source.

This illustrates why

$$D^*(x) = \frac{1}{2}$$

is the equilibrium discriminator.

Chapter 24

What Is Actually Learned?

It is tempting to say that the generator "learns to fool the discriminator." This is correct as an operational description, but the theoretical interpretation is deeper.

The generator changes its induced distribution:

$$p_g.$$

The discriminator measures differences between:

$$p_{\text{data}} \quad \text{and} \quad p_g.$$

The adversarial game therefore creates a feedback loop:

$$\boxed{G \to p_g \to D \to \text{gradient signal} \to G.}$$

As training progresses, the generator is encouraged to move its distribution toward the data distribution.

Chapter 25

Training versus Generation

Table 25.1: Training and generation in the original GAN framework

Training	**Generation**
Uses real training examples	Does not require real examples at generation time
Samples latent z	Samples latent z
Uses both G and D	Uses G only
Updates discriminator	No discriminator update is needed
Updates generator	Generator is used as the learned sampler
Learns the adversarial game	Produces samples from p_g

This is one of the major practical attractions of GANs: once G has been trained, generation requires only

$$\boxed{z \sim p_z, \qquad x = G(z).}$$

Chapter 26

Important Conceptual Distinctions

26.1 The discriminator is not the final model

The discriminator is a training mechanism.

After training, the generator is the component used to generate samples.

26.2 The discriminator does not directly improve the image

The discriminator produces a scalar score:

$$D(G(z)).$$

Backpropagation propagates information from this score through D and then through G. Therefore the generator parameters are updated indirectly.

26.3 The generator does not directly see the real image as a target

Unlike supervised regression, the original generator is not trained with a direct target image:

$$G(z) \neq x_{\text{real}}$$

as a pointwise reconstruction objective.

Instead, it receives an adversarial signal from D.

26.4 GANs learn distributions, not individual examples

The theoretical objective is expressed in terms of

$$p_{\text{data}} \quad \text{and} \quad p_g.$$

Thus the goal is not to reproduce a particular training example, but to make the overall generated distribution resemble the data distribution.

Chapter 27

Relation to the Broader Generative-Model Picture

Generative models attempt to capture structure in data rather than only predicting labels.

A GAN is therefore a generative framework:

$$\boxed{z \to x.}$$

A discriminative model typically learns:

$$\boxed{x \to y.}$$

The discriminator inside a GAN is discriminative, but the complete GAN is a generative model because its ultimate purpose is to learn a generator that produces samples.

27.1 Generative and discriminative roles coexist

This is an important conceptual point.

Inside the GAN:

$$D : \text{discriminative},$$

while

$$G : \text{generative}.$$

The adversarial framework combines them into one training system.

Chapter 28

Key Equations to Memorize

1. Generator

$$\boxed{x = G(z), \qquad z \sim p_z.}$$

2. GAN value function

$$\boxed{V(D,G) = \mathbb{E}_{p_{\text{data}}}[\log D(x)] + \mathbb{E}_{p_z}[\log(1 - D(G(z)))].}$$

3. Minimax game

$$\boxed{\min_G \max_D V(D,G).}$$

4. Optimal discriminator

$$\boxed{D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.}$$

5. Virtual training criterion

$$\boxed{C(G) = V(D_G^*, G).}$$

6. Jensen–Shannon form

$$\boxed{C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \| p_g).}$$

7. Global optimum

$$\boxed{p_g = p_{\text{data}}.}$$

8. Optimal discriminator at equilibrium

$$\boxed{D(x) = \frac{1}{2}.}$$

9. Minimax generator loss

$$\boxed{L_G = \mathbb{E}_z[\log(1 - D(G(z)))].}$$

10. Non-saturating generator loss

$$\boxed{L_G^{NS} = -\mathbb{E}_z[\log D(G(z))].}$$

Chapter 29

Common Questions and Their Answers

29.1 Why does the discriminator become $1/2$?

Because at equilibrium:

$$pdata(x) = pg(x).$$

Therefore:

$$D^*(x) = \frac{pdata(x)}{pdata(x) + pg(x)} = \frac{p}{p+p} = \frac{1}{2}.$$

29.2 Why does the generator need a discriminator?

The generator does not have an explicit target image. The discriminator provides a learned measure of how distinguishable generated samples are from real samples.

29.3 Why does the discriminator need generated samples?

Because generated samples are the negative examples of its classification problem.

29.4 Why is the GAN objective a minimax game?

Because the discriminator wants to maximize its classification objective, while the generator wants to minimize it by making fake samples appear real:

$$\min_G \max_D V(D, G).$$

29.5 What does p_g mean?

It is the distribution induced by

$$G(z)$$

when

$$z \sim p_z.$$

It is not necessarily represented by an explicit tractable density formula.

29.6 What does the theoretical theorem actually prove?

In the idealized distribution-level setting, the global optimum is obtained exactly when

$$p_g = p_{\text{data}}.$$

It does not guarantee that a finite neural-network implementation trained with ordinary optimization will always reach that point.

29.7 Why does Jensen–Shannon divergence appear?

After substituting the optimal discriminator into the GAN objective, the resulting generator criterion algebraically becomes

$$-\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \| p_g).$$

Therefore minimizing the adversarial criterion is equivalent, at the idealized optimal-discriminator level, to minimizing the Jensen–Shannon divergence between the data and generated distributions.

29.8 Why can GAN training still be difficult if the theory is so clean?

The theorem assumes idealized conditions. Real training has finite neural networks, non-convex parameter spaces, minibatch estimates, imperfect discriminator optimization, and alternating updates. The theoretical optimum therefore does not imply simple or guaranteed optimization dynamics.

Chapter 30

Final Conceptual Summary

The original GAN paper can be remembered through five ideas.

1. **Generator:**
$$z \to G(z)$$
converts simple latent noise into synthetic samples.

2. **Discriminator:**
$$x \to D(x)$$
estimates whether a sample is real or generated.

3. **Adversarial game:**
$$\min_G \max_D V(D, G).$$

4. **Optimal discriminator:**
$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

5. **Global distribution-level solution:**
$$\boxed{p_g = p_{\text{data}}, \qquad D(x) = \frac{1}{2}.}$$

The deepest conceptual picture is therefore:

Generator proposes samples

↓

Discriminator detects the difference

↓

Generator receives a gradient showing how to improve

$$\downarrow$$

$$\boxed{\text{The generated distribution moves toward the data distribution}}$$

and, in the idealized limit,

$$\boxed{p_g = p_{\text{data}}.}$$

Chapter 31

Conclusion

The original *Generative Adversarial Nets* paper introduced a fundamentally different way of learning generative models. Instead of directly constructing a tractable likelihood, it defines a game between a generator and discriminator.

The discriminator solves a binary classification problem:

$$\text{real} \quad \text{vs.} \quad \text{generated.}$$

The generator uses the discriminator's feedback to improve its samples.

The mathematical structure is particularly elegant. For a fixed generator, the optimal discriminator is

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substituting this into the objective produces

$$C(G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \| p_g).$$

Since Jensen–Shannon divergence is non-negative and equals zero only when the two distributions are equal, the theoretical global solution is

$$\boxed{p_g = p_{\text{data}}.}$$

At that point,

$$\boxed{D(x) = \frac{1}{2}}$$

because real and generated samples have identical distributions.

The paper's contribution is therefore not merely the introduction of two neural networks. Its deeper contribution is the formulation of generative-model learning as an adversarial two-player optimization problem with a clear distribution-level theoretical interpretation.

Appendix A

Notation Reference

Symbol	Meaning
x	Data sample.
z	Latent noise vector.
$p_z(z)$	Prior/noise distribution from which z is sampled.
G	Generator.
$G(z)$	Generated sample.
θ_g	Generator parameters.
D	Discriminator.
$D(x)$	Probability-like discriminator output that x is real.
θ_d	Discriminator parameters.
p_{data}	True data-generating distribution.
p_g	Distribution induced by $G(z)$ for $z \sim p_z$.
$V(D, G)$	GAN minimax value function.
D^*_G	Optimal discriminator for a fixed generator.
$C(G)$	Virtual training criterion after optimizing D.
D_{KL}	Kullback–Leibler divergence.
JSD	Jensen–Shannon divergence.
k	Number of discriminator updates per generator update in Algorithm 1.
m	Minibatch size.

Appendix B

Paper Reference

Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. *Generative Adversarial Nets.* Advances in Neural Information Processing Systems 27, 2014, pp. 2672–2680.

Official paper: https://papers.nips.cc/paper_files/paper/2014/file/5ca3e9b122f61f8f06494c97b1af
pdf